EdgeLoRA: An Efficient Multi-Tenant LLM Serving System on Edge Devices

Zheyu Shen, Yexiao He, Ziyao Wang, Yuning Zhang, Guoheng Sun, Wanghao Ye, Ang Li
University of Maryland, College Park
College Park, MD, USA
{zyshen,yexiaohe,ziyaow,yuning,ghsun,wy891,angliece}@umd.edu

arXiv:2507.01438v1 [cs.DC] 2 Jul 2025

Abstract

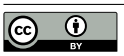
Large Language Models (LLMs) have gained significant attention due to their versatility across a wide array of applications. Fine-tuning LLMs with parameter-efficient adapters, such as Low-Rank Adaptation (LoRA), enables these models to efficiently adapt to downstream tasks without extensive retraining. Deploying fine-tuned LLMs on multi-tenant edge devices offers substantial benefits, such as reduced latency, enhanced privacy, and personalized responses. However, serving LLMs efficiently on resource-constrained edge devices presents critical challenges, including the complexity of adapter selection for different tasks, memory overhead from frequent adapter swapping. Moreover, given the multiple requests in the multi-tenant settings, processing requests sequentially will result in underutilization of computational resources and significant latency. This paper introduces EdgeLoRA, an efficient system for serving LLMs on edge devices in multi-tenant environments. EdgeLoRA incorporates three key innovations: (1) an adaptive adapter selection mechanism to streamline the adapter configuration process; (2) heterogeneous memory management, leveraging intelligent adapter caching and pooling to mitigate memory operation overhead; and (3) batch LoRA inference, which enables efficient batch processing to significantly reduce computational latency. Comprehensive evaluations using the Llama3.1-8B model demonstrates that EdgeLoRA significantly outperforms the status quo (*i.e.*, llama.cpp) in terms of both latency and throughput. The results demonstrates EdgeLoRA could achieve up to 4× boost in throughput with less energy consumption. Even more impressively, it manages to serve several orders of magnitude more adapters simultaneously without sacrificing inference performance. These results highlight EdgeLoRA's potential to transform edge deployment of LLMs in multi-tenant scenarios, offering a scalable and efficient solution for resource-constrained environments.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; • **Human-centered computing** → **Ubiquitous and mobile computing systems and tools**.

Keywords

Low-Rank Adaptation, Large Language Model, On-Device Serving



This work is licensed under a Creative Commons Attribution 4.0 International License.
MobiSys '25, Anaheim, CA, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1453-5/2025/06
https://doi.org/10.1145/3711875.3729141

ACM Reference Format:
Zheyu Shen, Yexiao He, Ziyao Wang, Yuning Zhang, Guoheng Sun, Wanghao Ye, Ang Li. 2025. EdgeLoRA: An Efficient Multi-Tenant LLM Serving System on Edge Devices. In *The 23rd Annual International Conference on Mobile Systems, Applications and Services (MobiSys '25), June 23–27, 2025, Anaheim, CA, USA.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3711875.3729141

1 Introduction

Large language models (LLMs) have emerged as transformative tools in natural language processing (NLP), excelling in diverse applications such as chatbots [2, 52], summarization [19, 60, 86], code generation [62, 67], and captioning [92]. Innovations from Anthropic [3], Google [78], Meta [27, 80], Microsoft [1], and OpenAI [9, 61] demonstrate their ability to reframe tasks like translation [8], classification [68], and question-answering [37] as language generation problems, achieving remarkable improvements. Beyond text, LLMs extend to image [17, 84], video [47], speech [71], and multimodal [93, 96] generation. Their success is driven by the Transformer architecture [81] and its variants [15, 51], which effectively model complex relationships in sequences, outperforming traditional methods [28]. These advancements position LLMs as foundational to modern AI, transforming research and applications across domains.

Although pretrained LLMs already demonstrate superior performance on general tasks, they can be further enhanced for domain-specific applications through fine-tuning. This adaptation process refines a pretrained LLM for optimal performance across diverse, specialized tasks. This pretrain-then-finetune paradigm has led to the proliferation of numerous fine-tuned variants of a single base LLM, each tailored to a specific task (*e.g.*, dialogue [29], writing [98], and code generation[62, 67]) or domain (*e.g.*, medical [44], mathematics [49, 76], and legal [94]). To address the computational overhead of fine-tuning, parameter-efficient methods like Low-rank adaptation (LoRA) have been developed [16, 33]. LoRA reduces computational demands by updating only a small portion of model parameters, which exploits the low dimensionality of parameter updates in fine-tuning and representing them with pairs of low-rank matrices, *i.e.*, LoRA adapters. Compared to fully fine-tuning, LoRA can reduce the number of training parameters by $10,000\times$ while maintaining comparable performance [33].

The use of multi-tenant LLM applications on edge devices has grown significantly, spanning diverse domains such as personalized virtual assistants [45], real-time translation tools [73], context-aware chatbots [2, 52], and on-device content moderation [59, 98]. These applications highlight the demand for flexible, efficient, and scalable serving strategies tailored for edge environments. To enable personalization and task-specific optimization, one promising

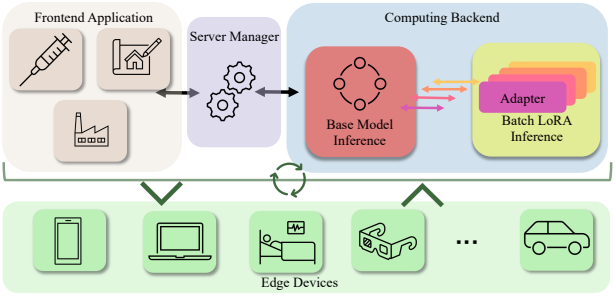


Figure 1: Multi-tenant LLM Serving on Edge Devices.

solution involves serving a set of LoRA models fine-tuned on a shared base LLM on edge devices, as illustrated in Figure 1. As presented in LoRA [33], merging LoRA adapters into the pretrained weights eliminates additional latency overhead during inference. However, serving multiple LoRA models concurrently introduces complexity, *i.e.*, adapters can be swapped by adding or subtracting LoRA matrices from the base model, but this approach incurs significant latency overhead and drastically reduces throughput. Alternatively, inference can proceed without merging by processing the base model and LoRA adapters in parallel, though this method also presents inefficiencies. Despite these approaches, serving multiple LoRA adapters with a shared base model on edge devices remains challenging: (1) users must manually select the appropriate adapter from a large pool, a process that is cumbersome and prone to errors; (2) efficient memory management is essential, particularly for swapping adapters between memory and disk to minimize resource overhead while increasing the number of adapters hosted by a single device; and (3) unmerged LoRA inference incurs substantial latency, as the parallel computation capabilities of edge devices are often underutilized when managing dynamic workloads. These challenges are particularly pronounced in the context of multi-tenant edge devices, which must efficiently manage multiple fine-tuned models concurrently.

Status Quo and Its Limitations. Current LLM serving systems, such as vLLM [38] and SGLang [99], are primarily designed to serve pretrained models efficiently. Some approaches, like SLoRA [70], mitigate memory constraints by dynamically swapping LoRA adapters between memory and disk, avoiding the need to load all adapters into memory simultaneously. Similarly, dLoRA [88] reduces latency by scheduling merged and unmerged LoRA inference operations. Other solutions, such as Punica [13], improve GPU utilization by batching pretrained weights during LoRA inference to enable parallelism. Despite these advancements, most existing solutions require users to manually select appropriate adapters, presenting a significant obstacle for users unfamiliar with the capability of adapters. Moreover, these systems fail to address the distinct challenges posed by multi-tenant edge devices. Unlike server environments, edge devices operate under severe resource constraints, highly dynamic workloads, and require support for heterogeneous system architectures, such as CPU, Metal, or BLAS-based backends. One serving framework tailored for edge devices is llama.cpp [24], which provides versatile support for multiple computing backends. However, llama.cpp lacks efficient support for LoRA inference, as it can only process requests that use the same adapters simultaneously. This limitation restricts its applicability in multi-tenant scenarios, where diverse adapters must be managed concurrently. These limitations highlight the pressing need for an efficient LLM serving system specifically designed to address the unique challenges of multi-tenant edge environments.

Efficiently serving LLMs on multi-tenant edge devices presents challenges such as selecting LoRA adapters, managing memory overhead, and optimizing efficiency. Specifically, adapter selection can be complex, as it requires accurately matching user requests to suitable adapters from potentially large and diverse adapter pools, making manual selection cumbersome and error-prone. Memory management becomes critical due to the limited resources available on edge devices; swapping adapters between memory and disk must be handled carefully to prevent throughput degradation and excessive latency. Additionally, optimizing computational efficiency is challenging, given that unmerged LoRA inference operations can lead to significant latency, particularly when parallel processing capabilities are underutilized, resulting in suboptimal throughput on dynamically changing workloads.

Overview of the Proposed Approach. In this work, we introduce EdgeLoRA, an efficient multi-tenant LLM serving system designed specifically for edge devices to address the key challenges associated with serving multiple LoRA adapters. To eliminate the need for users to manually specify adapters while maintaining performance, EdgeLoRA employs an adaptive adapter selection to automatically identify and deploy the optimal adapter based on request-specific requirements and the availability of adapters in the memory cache. To reduce the overhead associated with the swapping of adapters on resource-constrained edge devices, EdgeLoRA incorporates a heterogeneous memory manager that utilizes both the memory cache and a pre-allocated memory pool for efficient adapter management. Furthermore, EdgeLoRA introduces batch LoRA inference, a method that combines inference for pretrained weights and LoRA adapters into a unified process, significantly improving the utilization of computational resources under dynamic workloads. By combining these components, EdgeLoRA ensures efficient resource utilization, reduced inference latency, and robust adaptability for multi-tenant edge environments.

System Implementation and Evaluation Results. We implemented EdgeLoRA with over 1k+ lines of C++ code, extending the llama.cpp framework. For evaluation, EdgeLoRA was tested by serving Llama3.1-8B, Llama3.2-3B, and OpenELM-1.1B on three representative edge devices: Jetson AGX Orin, Jetson Orin Nano and Raspberry Pi 5. Experimental results demonstrate that EdgeLoRA significantly outperforms the state-of-the-art inference library for edge devices, llama.cpp. Specifically, EdgeLoRA achieves throughput improvements of 2-4× across a variety of tasks while increasing the number of concurrently served adapters by several orders of magnitude, all without compromising inference performance.

Summary of Contributions. The key contributions of EdgeLoRA are summarized as follows:

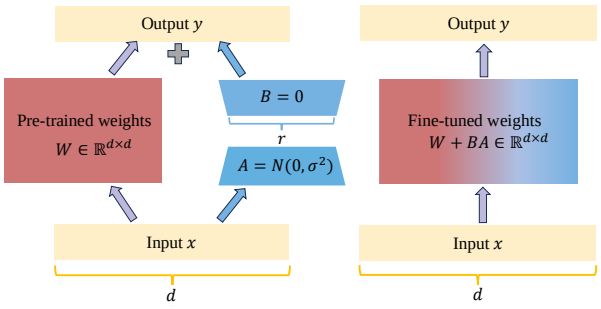
- EdgeLoRA introduces an adaptive adapter selection mechanism that dynamically selects the most appropriate adapter

based on incoming request requirements and memory availability, reducing manual intervention and adapter switching overhead while ensuring robust task performance.
- To optimize memory usage on edge devices, EdgeLoRA employs a hybrid memory management strategy that combines caching and pre-allocated memory pools, minimizing allocation overhead and reducing latency.
- EdgeLoRA improves GPU utilization by batching requests with different adapters, enabling simultaneous computation for base model inference and adapter-specific weights, thereby enhancing throughput and efficiency.

2 Background and Motivation

2.1 Low-Rank Adptation

LoRA [12, 16, 33] is a parameter-efficient fine-tuning (PEFT) [54] technique that enables the adaptation of LLMs to new tasks without requiring full model re-training. The motivation for developing LoRA arises from the observation that weight updates during adaptation exhibit a low intrinsic dimensionality. Specifically, as Figure 2(a) illustrates, LoRA retains the pre-trained base model weights $W \in \mathbb{R}^{d\times d}$ in a frozen state and augments each layer with trainable low-rank matrices $A \in \mathbb{R}^{r\times d}$ and $B \in \mathbb{R}^{d\times r}$ during the training phase, where $r \ll d$. This approach enables a significant reduction in the number of trainable parameters, thereby substantially decreasing memory consumption.



(a) LoRA fine-tuning. (b) LoRA inference.

Figure 2: The workflow of LoRA.

Compared to traditional full-parameter fine-tuning approaches, LoRA achieves a reduction in trainable parameters by several orders of magnitude (up to a 10,000×) while maintaining comparable performance in terms of model accuracy. During the inference phase, as illustrated in Figure 2(b), LoRA merges the product of the matrices $B \times A$ with the original weight matrix W, effectively incorporating the weight updates ΔW without incurring additional computational overhead. This characteristic distinguishes LoRA from earlier adapter-based methods [31] and prompt-tuning [40], which often introduce extra latency during inference. Due to its significant reduction in training and weight storage costs, LoRA has been widely adopted by the research community. Furthermore, LoRA has been employed extensively to enhance the capabilities of LLMs, such as in applications involving long-sequence modeling [14] and multimodal input [26].

2.2 Serving LLM on Edge Devices

Most LLMs are built upon the Transformer architecture [81], with the number of parameters ranging from several billion to several trillion [9, 34, 80]. These models aim to predict the next token conditioned on all preceding tokens, operating via an autoregressive generation process. During inference, tokens are iteratively generated based on the initial prompt and previously generated tokens until an end-of-sequence marker is reached. This autoregressive nature, coupled with the large parameter size, results in two key challenges for LLM inference: 1) variable inference latency, which depends on both input and output sequence lengths; and (2) high memory consumption, due to the need to maintain intermediate states for each active request.

The majority of popular LLM serving frameworks, such as vLLM [38], Text Generation Inference [20], and DeepSpeed-MII [57], are designed primarily for x86 operating systems and CUDA backends. However, edge devices are characterized by diverse operating systems and computational backends, such as CPU, Metal, and BLAS. The llama.cpp framework [24] provides a versatile LLM serving solution, predominantly written in C++, and supports multiple backends through the use of the GGML tensor library [25]. llama.cpp addresses the challenge of dynamic incoming requests using a slot state machine, which groups all available tokens across requests into a single batch, thereby enabling parallel processing of requests. Additionally, llama.cpp mitigates memory consumption by employing model quantization techniques, making it one of the most popular LLM serving frameworks for edge devices.

2.3 Fine-Tuned Adapters for Specialized Applications

Pre-trained LLMs [9, 27, 34] have demonstrated remarkable capabilities in solving a variety of general tasks by leveraging their extensive pretraining on large and diverse datasets. These models are proficient at capturing the nuances of language, encoding semantic relationships, and handling a broad spectrum of use cases. However, despite their general-purpose strengths, pre-trained models often fall short when applied to specialized domains requiring unique language, terminology, or domain-specific knowledge.

To bridge this gap, fine-tuning pre-trained models on targeted datasets tailored to specific applications has become a widely adopted approach. For example, Writing-Alpaca-7B [98], fine-tuned from LLaMA-7B [80] using a writing instruction dataset (an extension of the EDITEVAL [18] benchmark), significantly enhances LLaMA's performance in writing tasks. Writing-Alpaca-7B consistently outperforms larger off-the-shelf LLMs in tasks requiring advanced writing assistance. Similarly, ChatDoctor [44] is based on the fine-tuned LLaMA-7B [80] model, utilizing the Alpaca instruction dataset [77] combined with the HealthCareMagic100k patient-doctor dialogue dataset. This fine-tuning enables ChatDoctor to better understand patient concerns and deliver informed, domain-specific advice, surpassing the performance of generic LLMs in medical consultation tasks.

However, a key challenge associated with the fine-tuning process is the trade-off between generalization [5] and specialization. Fine-tuning a model for a specific domain often leads to a decline in its performance on tasks outside that domain. For example,

Table 1: OpenMath2-8B outperforms Llama3.1-8B-Instruct on math related tasks. But it sacrifices on other general-purpose tasks.

Task	Llama3.1-8B-Instruct	OpenMath2-8B
GSM8K	84.5	91.7
MATH	51.9	67.8
MMLU	68.2	36.9
MMLU-PRO	37.9	16.3
IFEVAL	41.8	17.2

OpenMath2-8B [79], fine-tuned on Llama3.1-8B-Base [27] using the OpenMathInstruct-2 [79] dataset, achieves exceptional performance in mathematical tasks. In contrast, Llama3.1-8B-Instruct [27] also fine-tuned on Llama3.1-8B-Base using the general-purpose datasets, maintaining a broader range of capabilities. As shown in Table 1, OpenMath2-8B surpasses Llama3.1-8B-Instruct on the whole MATH benchmark [30] by 15.9%. However, this improvement comes at the expense of reduced general-purpose capabilities compared to the pretrained model. This highlights the challenge of identifying a universally optimal adapter for serving LoRA models. Instead, adapter selection must be carefully tailored to align with the specific requirements of the application, ensuring that the chosen adapter delivers the desired balance between specialization and generalization.

3 System Design

3.1 Overview

EdgeLoRA introduces an efficient multi-tenant serving system specifically designed for deploying LLMs with multiple LoRA adapters on resource-constrained edge devices. The system addresses three critical challenges: dynamically selecting suitable adapters, efficiently managing adapter memory, and improving the efficiency of LoRA inference for concurrent requests. Figure 3 provides a high-level view of the EdgeLoRA. As our implementation is based on llama.cpp, EdgeLoRA shares a similar design structure, comprising two main components: the **Server Manager** and the **Computing Backend**. The Server Manager oversees request handling and memory management, with its core functionality driven by the **Slot State Machine** (§4), which manages concurrent requests. This component is crucial for dynamically allocating system resources, selecting appropriate adapters, and ensuring efficient operations under multi-tenant workloads. Specifically, the Server Manager includes two key modules: **Adaptive Adapter Selection** (§3.2), responsible for intelligent adapter selection, and the **Adapter Memory Manager** (§3.3), which ensures automatic adapter selection and efficient memory usage. Once requests are grouped into batches and the required LoRA adapters are loaded into the memory cache, the **Computing Backend** constructs and executes the computational graph. The Computing backend integrates the base model inference with **Batched LoRA Inference** (§3.4) to optimize the resource utilization and hence improve efficiency.

3.2 Adaptive Adapter Selection

Selecting the optimal LoRA adapter for a given request poses significant challenges due to the variability in application requirements and the computational overhead of loading and switching adapters. To address these challenges, EdgeLoRA introduces an adaptive adapter selection mechanism. This mechanism dynamically identifies and deploys the optimal adapter based on application-specific needs and the availability of adapters in the memory cache. As illustrated in Figure 4, the adaptive selection mechanism operates as follows: (1) when a new request is received, the system first checks if a specific adapter ID has been provided. If an adapter is explicitly specified, it is directly employed, bypassing the adaptive selection process. (2) Otherwise, the adaptive selection mechanism is employed to choose the most suitable adapter by analyzing the incoming prompt and the availability of adapters in the memory cache. The detailed steps of this mechanism are outlined in Algorithm 1.

Algorithm 1: Adaptive Adapter Selection

Input: User prompt x, Memory cache M, Adapter set A, Evaluation datasets $D = \{d_1, d_2, \ldots, d_m\}$, Adapter router C
Output: Selected Adapter a^*

```
if Adapter a is explicitly specified for request x then
    return a ;                    // Bypass adaptive selection
if Adapter router C is not available then
    foreach dataset d_i ∈ D do
        foreach adapter j ∈ A do
            Evaluate performance P_ij for adapter j using dataset d_i;
    Train adapter router C with prompt x as input and performance P_ij for adapter j using dataset d_i as output;
Compute confidence scores S = {s_1, s_2, ..., s_n} using adapter router C for prompt x;
A' ← Top-k adapters from A based on scores in S;
foreach adapter a' ∈ A' in descending order of confidence do
    if a' ∈ M then
        return a' ;               // Adapter is available
Load the adapter with the highest score from A' into memory cache M;
return adapter a* from A';
```

Profiling-Based Adapter Selection. EdgeLoRA employs a profiling-based method to generate training data for an adapter router, wherein the performance of each adapter is evaluated using diverse public evaluation datasets. Let $D = \{d_1, d_2, \ldots, d_m\}$ denote the set of evaluation datasets, and for each dataset d_i, the system determines the performance P_{ij} for each adapter j. The best-performing adapters for each dataset are identified, and this information is used to train a multi-label classifier that serves as the adapter router. In the adapter router, the user-provided prompt x serves as the input, and the output comprises scores $s_j \in [0, 1]$ for each adapter j, indicating its suitability for the given request. After training on such

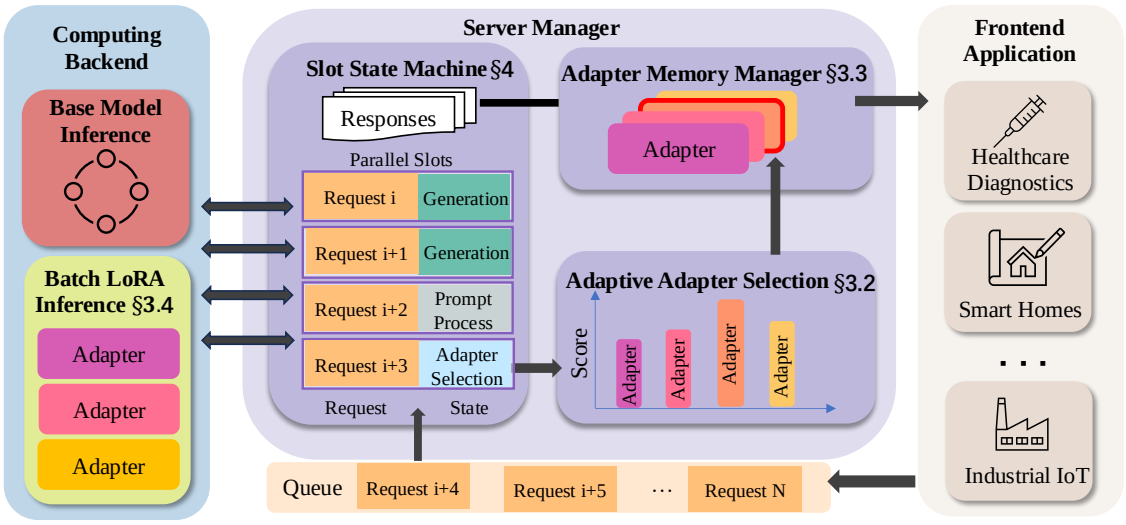


Figure 3: Overview of EdgeLoRA design.

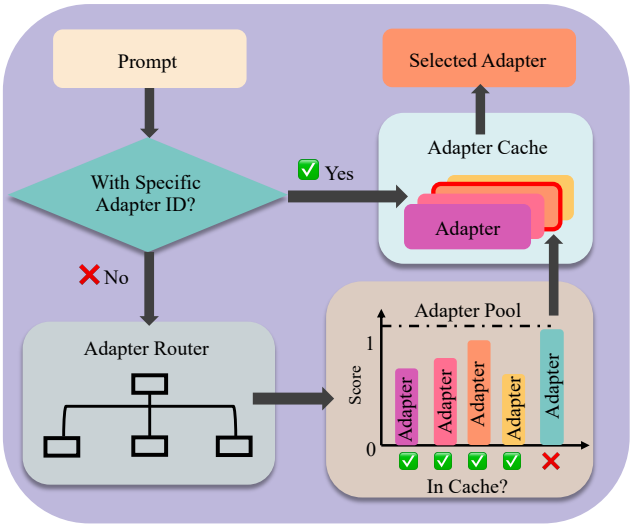


Figure 4: The workflow of adaptive adapter selection.

profiling data, the adapter router can assign a score to each adapter based on the given prompt, thereby enabling informed decisions regarding adapter selection. Specifically, the adapter with the highest score, $\text{argmax}_j(s_j)$, will be chosen as the most suitable when considering the quality of the response context. To further optimize resource utilization, the system selects a subset of top-scoring adapters, $A' \subset A$, where A is the full set of adapters. The selected adapters are then checked for availability in the memory cache in descending order of confidence score. If any of these adapters are already in the cache, they are immediately employed for inference. If none of the selected adapters is cached, the adapter with the highest score is dynamically loaded for use.

This adaptive approach addresses the challenge of manually identifying the best adapters for LLM requests, especially in multi-tenant edge environments. By automating the selection process, it significantly reduces the overhead associated with switching adapters between memory and disk. This approach maximizes memory utilization while ensuring that high-confidence adapters are prioritized.

3.3 Heterogeneous Memory Managemer

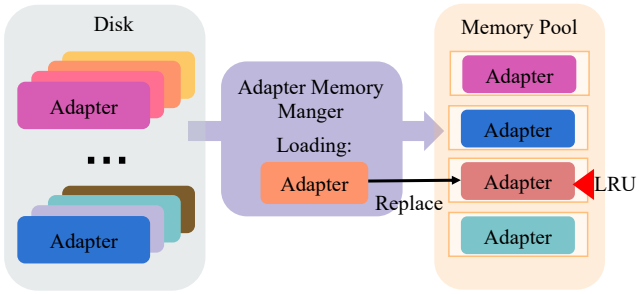


Figure 5: The adapter memory manager evicts the least frequently used adapter and loads the newly required one into a free memory block in the pool.

Edge devices often operate under strict memory constraints, posing significant challenges for serving LLM with multiple adapters concurrently. To address this limitation, EdgeLoRA incorporates a heterogeneous memory manager that utilizes both a memory cache and pre-allocated cache pools to effectively manage memory utilization. Figure 5 illustrates the comprehensive architecture of Heterogeneous Memory Management. Traditional LLM inference systems load all adapters into memory simultaneously and activate

them as needed during computation. However, our approach optimizes memory usage by loading adapters only when required, thus significantly reducing the memory overhead.

To minimize the frequent swapping of adapters between memory and disk, the system employs a memory cache C to store frequently accessed adapters. Let $C = \{a_{c_1}, a_{c_2}, \ldots, a_{c_l}\}$ be the set of cached adapters, where $l \leq k$. When the cache reaches its capacity, a replacement policy is used to evict the least frequently used and load the newly required adapter. This replacement policy aims to maximize the cache hit rate, defined as $H = \frac{h_{\text{cache}}}{h_{\text{total}}}$, where h_{cache} is the number of adapter requests successfully served from the cache, and h_{total} is the total number of adapter requests.

To further reduce runtime memory allocation overhead, the heterogeneous memory manager employs a pre-allocated memory pool $P = \{p_1, \ldots, p_l\}$, consisting of fixed memory blocks. Each memory block $p_i \in P$ corresponds to the size of a single adapter in the memory cache. When a new adapter is required, it is assigned to a free block p_i, avoiding dynamic memory allocation during runtime. This approach minimizes memory fragmentation, reduces the latency associated with memory allocation and deallocation, and enhances system stability under dynamic workloads. By combining memory caching and pre-allocated memory pools, the heterogeneous memory manager ensures efficient memory utilization while maintaining low latency and high system stability. When used in cooperation with the adaptive adapter selection mechanism, this memory management approach significantly reduces memory overhead and ensures efficient operation in resource-constrained edge environments.

3.4 Batch LoRA Inference

Different from merged LoRA inference as illustrated in Figure 2(b), unmerged LoRA inference separates the computations of the pre-trained LLM weights and LoRA adapter weights, allowing for more modular and dynamic computation during inference. Requests that require the same adapter can be grouped together, enabling shared computation for both the pre-trained LLM and LoRA adapter weights. However, in practical multi-tenant environments, requests requiring the same adapter are relatively rare and dynamic workloads require simultaneous use of multiple LoRA adapters for diverse tasks, limiting opportunities to group them effectively during unmerged LoRA inference. To address this limitation, we propose a novel approach named Batch LoRA Inference, which enables requests with different adapters to be processed within a single batch, thereby maximizing computational efficiency.

Figure 6 further illustrates batching LoRA inference. Consider a scenario with multiple distinct requests $\{x_0, \ldots, x_n\}$, each requiring a unique LoRA adapter. During inference, the inputs for all requests are batched together, and the computations involving the pre-trained weights are performed in parallel, represented as $W \times [x_0, x_1, \ldots, x_n]$. To further optimize efficiency, requests requiring the same adapter are grouped into a unified batch (u-batch) for the computation of the LoRA components. This ensures that shared computations for the same adapter are processed efficiently, leveraging the parallelism of modern GPUs. Once the computations involving the LoRA part are complete, the results from both the pre-trained part and the LoRA adapters are combined. Finally,

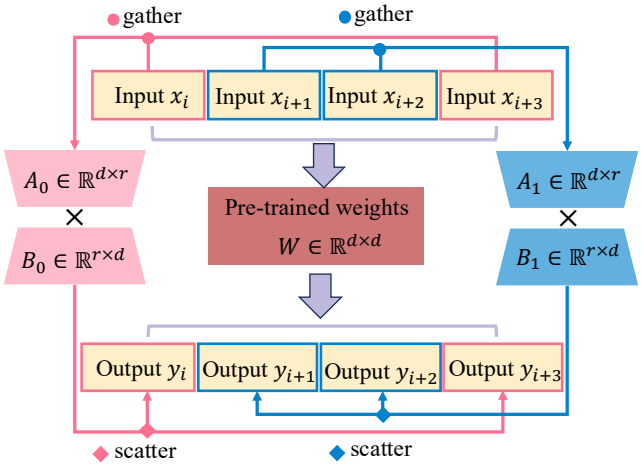


Figure 6: Batch LoRA inference.

the aggregated outputs are scattered back to their original locations in the input batch, yielding the final results, $[y_0, y_1, \ldots, y_n] = W \times [x_0, x_1, \ldots, x_n] + [B_0A_0x_0, B_1A_1x_1, \ldots, B_nA_nx_n]$.

By batching requests together, the system is able to exploit the inherent parallelism of modern GPU architectures, thus reducing per-request latency and improving throughput. This improvement is especially beneficial in scenarios involving limited computing resources, where maintaining large batch sizes is critical for maximizing the utilization of computing resources. In addition, the ability to handle multiple adapters in a single batch allows for greater flexibility and adaptability, ensuring that computational resources are utilized efficiently in multi-tenant environments with diverse and dynamic workloads.

4 Implementation

This section represents the implementation of EdgeLoRA based on llama.cpp. EdgeLoRA consists of two key components: the Server Manager, which acts as the server front-end, and the Computing Backend, which handles inference operations. Together, these components address challenges related to adapter selection, memory management, and efficient LoRA inference. The Server Manager uses a slot state machine to manage multiple requests concurrently, as illustrated in Figure 7. When a new request arrives, it is allocated to an idle slot, which transitions through the following states: (1) **Adapter Selection**, where Algorithm 1 determines the optimal adapter for the request; (2) **Prompt Processing**, where the designated adapter is used to decode the input prompt; and (3) **Generation**, where tokens are decoded iteratively to generate the response. This design ensures efficient request handling while maintaining optimal resource utilization. The Computing Backend processes the requests in batches forwarded by the Server Manager. It constructs a computational graph that integrates both base model inference and Batch LoRA Inference, optimizing computation efficiency for dynamic multi-adapter workloads.

The implementation of EdgeLoRA overcomes three primary challenges, including: (1) implementing a memory-efficient adapter router for precise task identification; (2) minimizing adapter-swapping

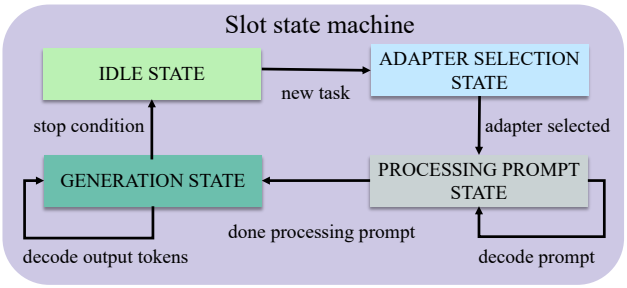


Figure 7: Slot state machine in our Server Manager.

overhead by implementing a robust cache management strategy; and (3) leveraging GPU parallelism during LoRA inference to maximize throughput. The system is implemented in over 1k+ lines of C++ code, with 500 lines of JavaScript simulating the front-end application.

4.1 Adaptive Adapter Selection

EdgeLoRA employs a memory-efficient adapter router fine-tuned with LoRA on the same base model deployed on edge devices. The router is implemented as a custom multi-label classifier using the HuggingFace Transformers Library [87]. A `Linear` layer is appended to the `LlamaModel`, with input dimensions corresponding to the model's `hidden_dim` and output dimensions representing the number of adapters. The loss function, `torch.nn.BCEWithLogitsLoss` [64], is used to train the router, with ground-truth labels indicating which adapters can generate correct responses for given prompts.

To train the router, evaluations are conducted for each LoRA adapter on five benchmarks using the Eleuther AI Language Model Evaluation Harness [23], including IFEval [100], BBH [75], MATH [30], GPQA [66], and MMLU-PRO [83]. The evaluation results are used as ground truth for training, with prompts processed consistently with the evaluation harness.

In the adapter selection stage, the input prompt is processed by the adapter router, which consists of the shared base model and an additional `Linear` layer. Since the computational cost of the base model far exceeds that of the `Linear` layer, the overhead introduced by adaptive adapter selection is roughly equivalent to the time required for decoding the input prompt. This results in minimal additional computational overhead while enabling accurate adapter selection. Additionally, the adapter router efficiently leverages memory already occupied by the base model, incurring only negligible extra memory usage from the additional `Linear` layer.

4.2 Heterogeneous Memory Management

To minimize memory operation overhead while providing efficient and low-latency access to frequently utilized adapters, EdgeLoRA incorporates a heterogeneous memory management strategy combining a Least Recently Used (LRU) [65] memory cache and a pre-allocated memory pool.

The memory cache is implemented utilizing an LRU policy to manage adapters effectively. The LRU cache retains frequently accessed adapters in memory, evicting less-used adapters when the cache is full. This aligns with the unbalanced locality of adapters observed in real-world scenarios [46]. The adapter invocation probabilities exhibit a long-tail distribution, with approximately 10% of adapters accounting for roughly 80% of the invocation probability [42]. When adapter locality becomes more unbalanced, some adapters are used more frequently. As a result, the LFU cache could achieve a higher cache hit rate, further improving the overall system throughput. The implementation employs the C++ Standard Template Library (STL) [36], specifically leveraging `std::list` and `std::unordered_set` to implement the LRU policy. In the event that a new adapter must be loaded when the cache has reached its capacity, the least recently used adapter is evicted, and its resources are returned to the memory pool for future reuse. During server initialization, the memory cache is prefilled with random adapters.

To reduce runtime overhead and mitigate the latency associated with frequent dynamic memory allocation, a pre-allocated memory pool has been implemented. This memory pool is comprised of memory blocks that are reserved during system initialization. The memory pool is represented by `std::stack<std::shared_ptr<adapter»`, which keeps track of available memory blocks and enables efficient allocation and deallocation during runtime. By reusing these blocks, the system minimizes allocation latency and ensures stable memory management under dynamic workloads. The combined utilization of an LRU cache and a pre-allocated memory pool enables efficient memory usage and provides low-latency access in resource-constrained environments.

4.3 Batch LoRA Inference

This section presents the details of batching LoRA inference implementation. The baseline approach processes each sample independently for adapter-specific computations, while base model computations are batched across the entire input. Although batching base model weights improves computational efficiency, adapter-specific computations remain isolated for each sample, leading to suboptimal GPU utilization and higher latency, especially in diverse multi-adapter workloads.

To address these limitations, our proposed group LoRA computing batches computations for samples sharing the same adapter. The implementation maps adapter IDs to sample indices, gathers data into sub-batches, and performs LoRA-specific matrix multiplications in a single operation using optimized GPU routines. And the results will be scattered back to their original positions in the output tensor. Gather and scatter operations ensure data alignment, while maintaining efficiency for adapter-specific computations.

By batching both base model and adapter-specific computations, group LoRA computing fully exploits GPU parallelism, reducing redundant operations and minimizing per-sample processing overhead. This results in enhanced GPU utilization, lower latency, and higher throughput, particularly in prompt processing stage where multiple samples share the same adapter. This scalable approach is highly effective for real-world multi-adapter scenarios that demand efficient, low-latency processing.

5 Evaluation

We evaluate the performance of EdgeLoRA on synthetic workloads. Specifically, we evaluate the scalability of EdgeLoRA by serving up to two thousand LoRA adapters simultaneously and compare it with llama.cpp. We then perform ablation studies to verify the effectiveness of individual components. **Model.** We evaluate EdgeLoRA using three different models: Llama3.1-8B [27], Llama3.2-3B, and OpenELM-1.1B [55], which are popular open-source LLMs. Each model is paired with adapter, and quantization configurations are presented in Table 2. All LoRA adapters are quantized using the Q8_0 format [25]. While we use these models for evaluation, EdgeLoRA is flexible and compatible with other transformer-based architectures, such as GPT-3 [9], Phi3 [1], Mixtral MOE [34], and Qwen [4].

Table 2: Model, adapter, and quantization configurations.

Setting	Base model	LoRA rank	Quantization
S1	Llama3.1-8B	32	Q8_0
S2	Llama3.2-3B	16	Q4_0
S3	OpenELM-1.1B	16	Q4_0

Hardware. We conduct our experiments across various edge devices, including Jetson Orin Nano (mid-tier), Jetson Orin AGX (high-tier), and Raspberry Pi 5. The Jetson devices are equipped with GPUs, while the Raspberry Pi relies solely on its CPU. These devices have memory capacities ranging from 8GB to 64GB. Our results demonstrate that EdgeLoRA can efficiently serve thousands LoRA adapters on resource-constrained edge devices.

Baselines. Since llama.cpp is a comprehensive LLM serving system designed to support various edge devices and capable of simultaneously serving multiple LoRA models, and other frameworks either exclusively support server environments or do not support LoRA models like MLC-LLM [58], we compare several variants of EdgeLoRA against llama.cpp [24].

- llama.cpp is an LLM serving system implemented entirely in C++. It supports multiple computation backends, including CPU, GPU, and METAL. When serving multiple LoRA models, the system loads all LoRA models into memory during server initialization. Users can send requests to dynamically adjust the scaling of different LoRA models and deploy them for computation as needed.
- EdgeLoRA builds upon the full feature set of EdgeLoRA, incorporating dynamic adapter selection to automatically choose the most suitable adapter for each request.
- EdgeLoRA(w/o AAS) is a variant of EdgeLoRA where adaptive adapter selection is disabled, requiring all requests to manually specify an adapter.

Metrics. The performance of serving systems can be evaluated using several key metrics, including latency and throughput. Following common practice, we report *throughput*, *average request latency*, *average first-token latency*, and *SLO attainment*. SLO (Service Level Objective) attainment is defined as the percentage of requests that return the first token within 6 seconds. Additionally, we also evaluate *power consumption*, which provides a quantitative analysis of the energy usage of edge devices.

5.1 End-to-End Results on Synthetic Workloads

Workload trace.

We generate synthetic workload traces using a Gamma process to model the arrival intervals of requests. The total request rate across all adapters is R requests per second. For n adapters, the optimal adapter for requests are sampled according to a power-law distribution with exponent α, determining adapter locality. Specifically, the probability $P(i)$ of selecting adapter i with adapters sorted by frequency, is defined by $P(i) = \frac{i^{-\alpha}}{\sum_{j=1}^{n} j^{-\alpha}}$. This choice of a power-law distribution is motivated by the observed long-tail distribution of adapter invocation probabilities in real-world workloads [42]. A lower α leads to higher locality, meaning requests are concentrated on fewer adapters, while a higher α results in a more uniform distribution across adapters. Arrival intervals between consecutive requests follow a Gamma distribution characterized by a shape parameter ($1/cv^2$) and a scale parameter (cv^2/R), where the coefficient of variation (cv) controls workload skewness or burstiness. A higher cv introduces greater variability and burstiness in request arrival patterns. This approach accurately simulates dynamic workloads for benchmarking. To simulate real requests and process adapter selection, after EdgeLoRA invokes the adapter router, we generate k ordered adapters, denoted as A'. Also for EdgeLoRAand EdgeLoRA(w/o AAS), we set the number of slot as γ. Our tests evaluate various combinations of parameters, including γ, k, α, R, and cv. For each request, the input and output lengths are sampled from uniform distributions $U[I_l, I_u]$ and $U[O_l, O_u]$, respectively. By default, each trace lasts for 5 minutes. To conduct comprehensive experiments, we first pick a set of default parameters for generating workloads, as shown in Table 3.

Table 3: Default parameters for generating the synthetic workloads and server. "S1@AGX" means running S1 setting on Jetson AGX Orin.

Setting	γ	k	α	R	cv	$[O_l, O_u]$	$[I_l, I_u]$
S1@AGX	20	3	1	0.5	1	[8,128]	[8,256]
S2@AGX	50	3	1	0.6	1	[8,128]	[8,256]
S3@AGX	50	3	1	1	1	[8,256]	[8,256]
S2@Nano	5	3	1	0.3	1	[8,128]	[8,256]
S3@Nano	10	3	1	0.6	1	[8,128]	[8,256]
S3@Rasp	5	3	1	0.2	1	[8,128]	[8,128]

Comparison with llama.cpp. We compare EdgeLoRA and EdgeLoRA (w/o AAS) with llama.cpp for serving multiple LoRA adapters, with results presented in Table 4. Notably, EdgeLoRA can serve over 1,000 adapters simultaneously on Jetson Orin AGX, incurring minimal overhead as the number of LoRA models increases. In contrast, llama.cpp is limited to serving only 50 adapters due to memory constraints. Overall, EdgeLoRA achieves 2-4× the throughput higher than llama.cpp while serving a significantly larger number of adapters across three edge devices.

First token latency and SLO. We compare EdgeLoRA and EdgeLoRA (w/o AAS) with llama.cpp in terms of average first token latency and SLO attainment relative to the number of adapters. Table 5 shows that EdgeLoRA maintains a high SLO across all settings, even

Table 4: Throughput (req/s) comparison between llama.cpp, EdgeLoRA, and EdgeLoRA(w/o AAS) cross devices.

Setting	n	llama.cpp	EdgeLoRA	EdgeLoRA(w/o AAS)
S1@AGX	20	0.11	0.45	0.45
	50	0.11	0.44	0.44
	100	OOM	0.44	0.44
	1000	OOM	0.42	0.44
S2@Nano	20	0.12	0.26	0.27
	100	OOM	0.26	0.26
	500	OOM	0.25	0.26
S3@Rasp	20	0.05	0.19	0.20
	100	OOM	0.19	0.18
	200	OOM	0.18	0.18

when serving a large number of adapters. While the first token latency of EdgeLoRA is higher than that of EdgeLoRA(w/o AAS) due to the additional computation for adaptive adapter selection, this does not significantly impact the SLO as shown in Table 6, ensuring high user satisfaction. Additionally, we observe that the computational overhead introduced by adaptive adapter selection is roughly equivalent to the time spent on decoding the input prompts.

Table 5: SLO comparison between llama.cpp, EdgeLoRA, and EdgeLoRA(w/o AAS) on S3@Nano setting.

n	llama.cpp	EdgeLoRA	EdgeLoRA(w/o AAS)
20	1.11%	98.67%	100%
100	OOM	98.67%	100%
200	OOM	98.67%	100%
500	OOM	98.67%	100%
1000	OOM	98.00%	100%

Table 6: First token latency (s) comparison between llama.cpp, EdgeLoRA, and EdgeLoRA(w/o AAS) on S3@Nano setting.

n	llama.cpp	EdgeLoRA	EdgeLoRA(w/o AAS)
20	206.28	0.51	0.29
100	OOM	0.54	0.29
200	OOM	0.54	0.32
500	OOM	0.56	0.33
1000	OOM	0.58	0.35

Adapter Locality. Real-world workloads typically exhibit temporal locality, where certain adapters are accessed more frequently due to user behavior or popularity differences. Thus, using a low α creates synthetic workloads with high adapter locality [46], effectively simulating realistic scenarios. We compare EdgeLoRA with llama.cpp across different adapter locality scenarios under the S1@AGX setting with 50 adapters. Table 7 shows that the throughput of llama.cpp is not sensitive to adapter locality, as llama.cpp preloads all adapters into memory and only needs to update adapter weights upon switching adapters. Similarly, the throughput of EdgeLoRA remains unaffected by adapter locality due to its use of an LRU-based memory cache, which retains frequently accessed adapters and reduces adapter swapping. Table 8 indicates that the average request latency for llama.cpp decreases slightly under high adapter locality conditions, benefiting from fewer adapter weight switches. The average request latency of EdgeLoRA also decreases in high-locality scenarios, as the LRU cache achieves a higher hit rate, further minimizing latency.

Table 7: Throughput (req/s) comparison between llama.cpp, EdgeLoRA on S1@AGX($n = 50$) with different adapter locality.

α	llama.cpp	EdgeLoRA
0.5	0.11	0.45
0.75	0.11	0.45
1	0.11	0.44

Table 8: Average request latency (s) comparison between llama.cpp, EdgeLoRA on S1@AGX($n = 50$) with different adapter locality.

α	llama.cpp	EdgeLoRA
0.5	8.61	2.67
0.75	8.79	2.67
1	8.94	2.80

Workload skewness. Users of multi-tenant LLM applications often access services in irregular patterns, causing sudden spikes when multiple users request simultaneously or when a application triggers bursts of usage. High skewness better represents realistic workloads in LLM serving scenarios [46]. The intervals between consecutive requests follow a Gamma distribution. When $cv = 1$, the Gamma distribution simplifies to an exponential distribution, indicating moderate burstiness. A $cv > 1$ signifies significantly greater variability and pronounced bursts in request patterns. We compare EdgeLoRA with llama.cpp across various workload skewness scenarios under the S1@AGX setting with 50 adapters. Table 9 demonstrates that llama.cpp's throughput significantly decreases and its latency increases under high workload skewness. This degradation occurs because llama.cpp processes requests sequentially, causing delays when consecutive requests require different adapters. In contrast, EdgeLoRA experiences a modest reduction in throughput and increase in latency due to its enhanced capability for parallel request processing. These results indicate that EdgeLoRA maintains robust performance even under highly skewed workloads. Specifically, when $cv = 2$, some interval between consecutive requests exceeds the request processing time, resulting in similar performance for both llama.cpp and EdgeLoRA when waiting for request in a long time.

Power consumption. We compare EdgeLoRA with llama.cpp in terms of the power consumption across various settings. Power consumption data is collected using jetson-stats [6], a monitoring tool for system status on Jetson devices. We record power consumption every second and calculate the average across all requests.

Table 9: Throughput (req/s) comparison between llama.cpp, EdgeLoRA on S1@AGX(n = 50) with different workload skewness.

cv	llama.cpp	EdgeLoRA
1	0.11	0.44
1.25	0.10	0.23
1.5	0.09	0.11
2	0.03	0.03

Table 10: Average request latency (s) comparison between llama.cpp, EdgeLoRA on S1@AGX(n = 50) with different workload skewness.

cv	llama.cpp	EdgeLoRA
1	8.61	2.80
1.25	9.13	5.27
1.5	10.24	9.42
2	29.25	29.54

Table 11 shows that EdgeLoRA achieves higher energy efficiency than llama.cpp. Moreover, EdgeLoRA achieves higher throughput while requiring less computation time to process a given number of requests, resulting in greater power savings.

Table 11: Power consumption (Watt) comparison between llama.cpp and EdgeLoRA cross devices.

Setting	llama.cpp	EdgeLoRA
S1@AGX (n=20)	32.16	28.04
S2@AGX (n=50)	24.43	24.42
S2@Nano (n=20)	10.27	8.67

Comparison with own variants. Since llama.cpp cannot serve a large number of adapters, our comparison focuses primarily on EdgeLoRA and EdgeLoRA (w/o AAS). Figure 8 illustrates how both systems scale with the number of adapters. On both Jetson AGX Orin and Jetson Orin Nano, EdgeLoRA achieves similar throughput to EdgeLoRA (w/o AAS). This latency gap is due to EdgeLoRA leverages adaptive adapter selection to maximize the utilization of in-memory adapters. When more requests can utilize in-memory adapters, computational parallelism increases, allowing multiple requests to be processed simultaneously without waiting for adapter loading operations. As a result, EdgeLoRA exhibits lower latency than EdgeLoRA (w/o AAS) on both Jetson AGX Orin and Jetson Orin Nano. As the number of adapters increases, EdgeLoRA maintains stable throughput, and latency increases gradually. However, once the number of adapters exceeds a certain threshold, the latency stabilizes. This stability is due to efficient memory management, where the overhead of swapping adapters remains constant as the number of adapters grows. Consequently, EdgeLoRA can scale to handle a significantly large number of adapters without additional overhead, with the only constraint being disk capacity.

5.2 Adapter Router Performance

We use five key datasets to generate data for both training and testing. The datasets were selected from the Open LLM Leaderboard [87], as they assess a broad range of reasoning abilities and general knowledge across multiple domains. We randomly split the data, using 80% for training and the remaining 20% for testing. Detailed information about these datasets is provided below:

- **IFEval** [100] A dataset for testing a model's ability to follow explicit instructions like formatting or keyword inclusion. It focuses on adherence to instructions using strict, rigorous metrics.
- **BBH** [75] A subset of 23 challenging BigBench tasks using objective metrics to evaluate models. Tasks include multistep reasoning, language understanding, and world knowledge, offering insights into model capabilities.
- **MATH** [30] A dataset of high-school competition problems formatted in LaTeX and Asymptote. Only level-5 MATH questions are included, requiring specific output formats.
- **GPQA** [66] A knowledge dataset with questions from PhD-level experts in biology, physics, and chemistry. It is highly validated, gated for restricted access, and avoids plain text examples to minimize contamination.
- **MMLU-PRO** [83] A refined version of MMLU addressing noisy data and contamination. It increases difficulty with 10-choice questions and expert-reviewed content, providing a higher-quality assessment.

We collected six well fine-tuned models based on Llama3.1-8B-instruct [27] from the Huggingface Hub: Llama-Spark [1], Defne-llama3.1-8B [2], Hercules-6.1-Llama-3.1-8B [3], Llama3.1-8B-ShiningValiant2 [4], Llama-3.1-8B-German-ORPO [5], and Llama-3.1-SauerkrautLM-8b-Instruct [6]. The pretrained model is one of the most powerful LLMs nowadays. And each fine-tuned model excels at least in one task, outperforming the pretrained model. We chose to fine-tune the adapter router based on the same pretrained model. The model was trained for 3 epochs with a learning rate of 1e−5, using a linear learning rate scheduler and the AdamW optimizer. LoRA-specific parameters included a rank of 32, an alpha value of 64, a dropout rate of 0.05, and targeted layers as the Q, K, V, Down, and Up layers. The fine-tuning process took approximately 8 hours using two NVIDIA RTX A6000 Ada Generation GPUs. After fine-tuning, we evaluated the performance of the adapter router on each task with the test data. Table 12 shows that the adapter router consistently outperforms individual adapters by dynamically assigning prompts to the most suitable adapter. The performance ceiling of the adapter router is determined by the optimal adapter selection for each prompt, thus inherently constrained by the maximum performance of the individual adapters.

[1] https://huggingface.co/VAGOsolutions/Llama-3.1-SauerkrautLM-8b-Instruct
[2] https://huggingface.co/Eurdem/Defne-llama3.1-8B
[3] https://huggingface.co/Locutusque/Hercules-6.1-Llama-3.1-8B
[4] https://huggingface.co/ValiantLabs/Llama3.1-8B-ShiningValiant2
[5] https://huggingface.co/Nekochu/Llama-3.1-8B-German-ORPO
[6] https://huggingface.co/VAGOsolutions/Llama-3.1-SauerkrautLM-8b-Instruct

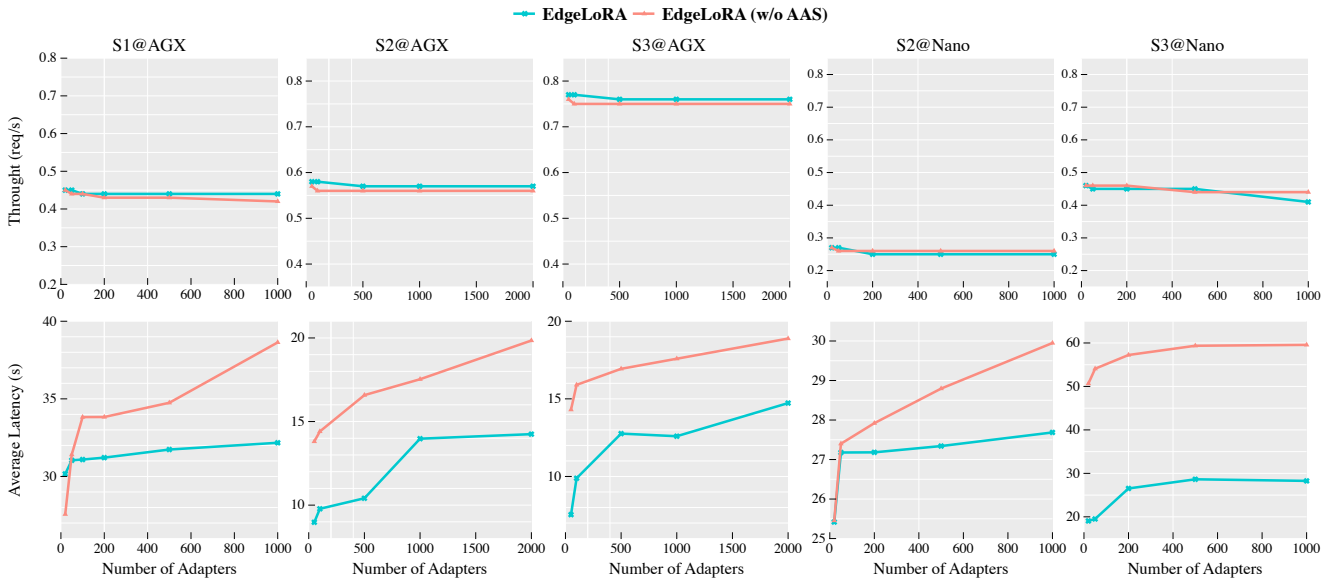


Figure 8: Throughput and average request latency of EdgeLoRA and EdgeLoRA (w/o AAS) under varying numbers of adapters. Both of them demonstrate scalability to a large number of adapters with similar throughput.

Table 12: Adapter router accuracy evaluated with LLaMA3.1-8B-Instruct.

Model	IFEval	BBH	MATH	GPQA	MMLU-PRO	Average
Llama-3.1-8B-Instruct	41.84	51.22	13.82	34.95	37.85	35.94
Llama-Spark	43.45	52.30	13.45	31.79	38.91	35.98
Defne-llama3.1-8B	40.92	53.10	14.56	32.42	38.82	35.96
Hercules-6.1-Llama-3.1-8B	47.13	51.09	13.54	32.63	37.42	36.36
Llama3.1-8B-ShiningValiant2	18.16	44.08	8.53	32.11	32.62	27.10
Llama-3.1-8B-German-ORPO	41.38	50.10	0.19	32.95	33.72	31.67
Llama-3.1-SauerkrautLM-8b-Instruct	45.52	51.85	15.40	33.16	39.57	37.10
Adapter Router (Our Approach)	46.22	53.60	13.82	38.74	38.74	38.22

5.3 Ablation Study

5.3.1 DVFS (Dynamic Voltage and Frequency Scaling) of Jetson. Jetson devices support multiple energy modes, allowing configuration of the number of active cores, their frequencies, and memory frequency to accommodate different power envelopes. Specifically, the Jetson Orin AGX supports TDPs (Thermal Design Power) of 50W, 30W, and 15W, while the Jetson Orin Nano offers 15W and 7W modes. Leveraging this functionality, we conducted experiments to evaluate how these devices perform in executing LLMs under varying power budgets and corresponding performance constraints. Table 13 presents the results of running S1, S2 and S3 on the Jetson Orin AGX under three different TDP levels. The results demonstrate that lower TDP levels constrain the throughput of the serving system.

Table 13: Throughput (req/s) on Jetson devices under different TDPs.

TDP	S1@AGX	S2@AGX	S3@AGX
50W	0.45	0.57	0.76
30W	0.31	0.51	0.24
15W	0.13	0.22	0.14

5.3.2 Number of slots. EdgeLoRA utilizes a slot state machine to handle concurrent requests. The number of slots can be manually configured to control how many requests are processed simultaneously, while additional requests are queued. A larger number of slots allows more requests to be processed concurrently. When slots are in the processing or generation states, they can be grouped into the same batch, leading to larger batch sizes. We conducted experiments to evaluate the impact of the number of slots on serving throughput. Table 14 presents the results for running S1, S2, and S3 on the Jetson Nano Orin with three different slot configurations. The results indicate that a larger number of slots enhances parallel computation capabilities.

Table 14: Throughput (req/s) on Jetson Orin Nano using different number of slots.

Number of slots	S2@Nano	S3@Nano
1	0.07	0.23
5	0.17	0.39
10	0.27	0.46
20	0.43	0.56

6 Related Work

LLM Serving Numerous studies have sought to optimize the efficiency of LLM serving. vLLM[38] introduced a memory-efficient solution based on PagedAttention, which effectively manages key-value cache memory using concepts inspired by classical virtual memory paging. This method significantly reduces memory fragmentation and enhances GPU memory utilization during LLM inference, achieving obviously throughput improvements. However, vLLM can only be deployed on server cluster. In contrast, PowerInfer[72] focuses on LLM serving using consumer-grade GPUs by implementing a GPU-CPU hybrid inference engine, aiming to enhance performance on personal computers rather than large-scale edge deployments. Both vLLM and PowerInfer do not explicitly address the efficiency challenges of LoRA inference. PipeInfer[11] employs speculative execution across distributed server clusters to reduce latency during token generation; however, this architecture is more suited for server-side environments and lacks emphasis on scenarios constrained by edge resources. Llumnix[74] addresses the challenges of scheduling to mitigate severe queuing delays and enhance load balancing in LLM serving, but its primary focus is on dynamic scheduling rather than optimizing efficiency specifically for LoRA adapters. InfiniGen [39] optimizes memory and computation during long-text generation using dynamic key-value cache management, but its reliance on speculative attention computations. LServe [91] efficiently accelerates long-sequence LLM serving using hybrid sparse attention. MLC-LLM [58] is a machine learning compiler and high-performance deployment engine for LLMs, which support multiple backend of edge devices. But it currently lacks support for serving LoRA models. Finally, Soter [69] offers a secure and efficient partitioning approach to safeguard model confidentiality on edge devices, yet its scope is limited to general neural networks rather than LLMs.

Efficient LoRA Inference The efficient inference of LoRA in LLMs has garnered considerable research attention. S-LoRA[70] was developed to serve thousands of concurrent LoRA adapters by managing them within unified memory through a unified paging system. Although S-LoRA enhances throughput, it overlooks the issue of high latency caused by unmerged LoRA during inference, which restricts its applicability in latency-sensitive settings. dLoRA[88] addresses this limitation by dynamically merging and unmerging adapters and migrating requests between worker replicas, thereby balancing throughput and reducing latency. Nevertheless, dLoRA does not fully resolve the challenge of accelerating unmerged LoRA inference and remains unsuitable for edge device deployment. V-LoRA[56] introduces an adaptive-tiling approach for batching LoRA adapters, enabling efficient computation of concurrent heterogeneous LoRA adapters. However, V-LoRA is specifically tailored to vision applications and does not generalize to broader LLM tasks. Punica[13] provides multi-tenant LoRA serving by batching the pretrained model weights, but it fails to consider batching for the LoRA adapters themselves, thereby limiting further efficiency gains. Moreover, Punica's architecture is designed primarily for shared GPU clusters, presenting challenges for deployment on edge devices. Joint compression [10] serves thousands of LoRA adapters efficiently by employing joint compression techniques, enabling scalability and high throughput while maintaining model performance. However, it would sacrifice some fine-tuned model performance, which is critical if precision or task-specific accuracy is crucial.

Optimize Edge LLM Serving with Algorithm Techniques Optimizing the serving of LLMs on edge devices edge devices has attracted significant attention due to inherent computational and memory constraints, prompting various algorithmic optimizations. Quantization techniques [22, 48, 90] significantly reduce memory requirements and computational overhead without substantially degrading accuracy. Pruning techniques have also been explored to enhance computational efficiency by eliminating redundant model parameters including structured methods [53, 89], and unstructured methods [21]. Knowledge distillation strategies [35, 82] transfer capabilities from larger models to compact, resource-friendly alternatives, enabling efficient inference. Moreover, innovative runtime optimizations such as split inference [7, 32] and speculative decoding [32, 95] further mitigate latency and bandwidth issues by intelligently distributing workload between cloud and edge components.

Parameter-efficient Fine-tuning Recent advances in PEFT for large pre-trained language models have demonstrated that updating only a small subset of parameters can yield performance on par with full fine-tuning. State-of-the-art PEFT techniques include LoRA [33], prefix-tuning [43], P-Tuning [50], prompt tuning [41], AdaLoRA [97], and IA^3 [63]. More recently, FLoRA [85] introduces a federated fine-tuning framework that leverages heterogeneous LoRA to enable privacy-preserving, distributed adaptation of LLM. Although our work focuses on LoRA, the same principles readily extend to other PEFT methods.

7 Conclusion

In this work, we presented EdgeLoRA, an efficient multi-tenant LLM serving system designed for edge devices to address the challenges of serving multiple LoRA adapters. By introducing adaptive adapter selection, heterogeneous memory management, and batched LoRA inference, EdgeLoRA eliminates manual adapter selection, optimizes memory usage, and improves computational efficiency. Our system achieves significant improvements in throughput, scalability, and energy efficiency, demonstrating its ability to manage dynamic workloads across multi-tenant edge environments. Experimental results confirm that EdgeLoRA outperforms existing solutions, supporting a larger number of adapters while maintaining low latency and high user satisfaction. These findings highlight the potential of EdgeLoRA to enable advanced, multi-tenant LLM applications on edge devices.

References

[1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Qin Cai, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Weizhu Chen, Yen-Chun Chen, Yi-Ling Chen, Hao Cheng, Parul Chopra, Xiyang Dai, Matthew Dixon, Ronen Eldan, Victor Fragoso, Jianfeng Gao, Mei Gao, Min Gao, Amit Garg, Allie Del Giorno, Abhishek Goswami, Suriya Gunasekar, Emman Haider, Junheng Hao, Russell J. Hewett, Wenxiang Hu, Jamie Huynh, Dan Iter, Sam Ade Jacobs, Mojan Javaheripi, Xin Jin, Nikos Karampatziakis, Piero Kauffmann, Mahoud Khademi, Dongwoo Kim, Young Jin Kim, Lev Kurilenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Yunsheng Li, Chen Liang, Lars Liden, Xihui Lin, Zeqi Lin, Ce Liu, Liyuan Liu, Mengchen Liu, Weishung Liu, Xiaodong Liu, Chong Luo, Piyush Madan, Ali Mahmoudzadeh, David Majercak, Matt Mazzola, Caio César Teodoro Mendes, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Liliang Ren, Gustavo de Rosa, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacroce, Shital Shah, Ning Shang, Hiteshi Sharma, Yelong Shen, Swadheen Shukla, Xia Song, Masahiro Tanaka, Andrea Tupini, Praneetha Vaddamanu, Chunyu Wang, Guanhua Wang, Lijuan Wang, Shuohang Wang, Xin Wang, Yu Wang, Rachel Ward, Wen Wen, Philipp Witte, Haiping Wu, Xiaoxia Wu, Michael Wyatt, Bin Xiao, Can Xu, Jiahang Xu, Weijian Xu, Jilong Xue, Sonali Yadav, Fan Yang, Jianwei Yang, Yifan Yang, Ziyi Yang, Donghan Yu, Lu Yuan, Chenruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219 [cs.CL] https://arxiv.org/abs/2404.14219

[2] Daniel Adiwardana, Minh-Thang Luong, David R So, Jamie Hall, Noah Fiedel, Romal Thoppilan, Zi Yang, Apoorv Kulshreshtha, Gaurav Nemade, Yifeng Lu, et al. 2020. Towards a human-like open-domain chatbot. *arXiv preprint arXiv:2001.09977* (2020).

[3] Anthropic. 2023. Claude 3 Model Card. https://www.anthropic.com/model-card-claude-3 Accessed: 2024-12-03.

[4] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. arXiv:2309.16609 [cs.CL] https://arxiv.org/abs/2309.16609

[5] Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. 2024. LoRA Learns Less and Forgets Less. arXiv:2405.09673 [cs.LG] https://arxiv.org/abs/2405.09673

[6] Raffaello Bonghi. 2023. Jetson-Stats. https://github.com/rbonghi/jetson_stats. Accessed: 2024-12-07.

[7] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin A Raffel. 2023. Distributed inference and fine-tuning of large language models over the internet. *Advances in neural information processing systems* 36 (2023), 12312–12331.

[8] Peter F Brown, John Cocke, Stephen A Della Pietra, Vincent J Della Pietra, Frederick Jelinek, John Lafferty, Robert L Mercer, and Paul S Roossin. 1990. A statistical approach to machine translation. *Computational linguistics* 16, 2 (1990), 79–85.

[9] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165

[10] Rickard Brüel-Gabrielsson, Jiacheng Zhu, Onkar Bhardwaj, Leshem Choshen, Kristjan Greenewald, Mikhail Yurochkin, and Justin Solomon. 2024. Compress then Serve: Serving Thousands of LoRA Adapters with Little Overhead. arXiv:2407.00066 [cs.DC] https://arxiv.org/abs/2407.00066

[11] Branden Butler, Sixing Yu, Arya Mazaheri, and Ali Jannesari. 2024. PipeInfer: Accelerating LLM Inference using Asynchronous Pipelined Speculation. *arXiv preprint arXiv:2407.11798* (2024).

[12] Arnav Chavan, Zhuang Liu, Deepak Gupta, Eric Xing, and Zhiqiang Shen. 2023. One-for-all: Generalized lora for parameter-efficient fine-tuning. *arXiv preprint arXiv:2306.07967* (2023).

[13] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. Punica: Multi-Tenant LoRA Serving. arXiv:2310.18547 [cs.DC] https://arxiv.org/abs/2310.18547

[14] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2023. Longlora: Efficient fine-tuning of long-context large language models. *arXiv preprint arXiv:2309.12307* (2023).

[15] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.

[16] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems* 36 (2024).

[17] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. arXiv:2010.11929 [cs.CV] https://arxiv.org/abs/2010.11929

[18] Jane Dwivedi-Yu, Timo Schick, Zhengbao Jiang, Maria Lomeli, Patrick Lewis, Gautier Izacard, Edouard Grave, Sebastian Riedel, and Fabio Petroni. 2022. EditEval: An Instruction-Based Benchmark for Text Improvements. arXiv:2209.13331 [cs.CL] https://arxiv.org/abs/2209.13331

[19] Wafaa S El-Kassas, Cherif R Salama, Ahmed A Rafea, and Hoda K Mohamed. 2021. Automatic text summarization: A comprehensive survey. *Expert systems with applications* 165 (2021), 113679.

[20] Hugging Face. 2023. Text Generation Inference: Large Language Model Text Generation Inference. https://github.com/huggingface/text-generation-inference.

[21] Elias Frantar and Dan Alistarh. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*. PMLR, 10323–10337.

[22] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).

[23] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation. https://doi.org/10.5281/zenodo.10256836

[24] Georgi Gerganov. 2023. llama.cpp: LLM inference in C/C++. https://github.com/ggerganov/llama.cpp.

[25] Georgi Gerganov. 2024. ggml. https://github.com/ggerganov/ggml. Accessed: 2024-12-03.

[26] Tao Gong, Chengqi Lyu, Shilong Zhang, Yudong Wang, Miao Zheng, Qian Zhao, Kuikun Liu, Wenwei Zhang, Ping Luo, and Kai Chen. 2023. Multimodal-gpt: A vision and language model for dialogue with humans. *arXiv preprint arXiv:2305.04790* (2023).

[27] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, and Others. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[28] Alex Graves and Alex Graves. 2012. Long short-term memory. *Supervised sequence labelling with recurrent neural networks* (2012), 37–45.

[29] Prakhar Gupta, Cathy Jiao, Yi-Ting Yeh, Shikib Mehri, Maxine Eskenazi, and Jeffrey P Bigham. 2022. InstructDial: Improving zero and few-shot generalization in dialogue through instruction tuning. *arXiv preprint arXiv:2205.12673* (2022).

[30] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874 [cs.LG] https://arxiv.org/abs/2103.03874

[31] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International conference on machine learning*. PMLR, 2790–2799.

[32] Chenghao Hu and Baochun Li. 2024. When the Edge Meets Transformers: Distributed Inference with Transformer Models. In *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 82–92.

[33] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[34] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG] https://arxiv.org/abs/2401.04088

[35] Yuxin Jiang, Chunkit Chan, Mingyang Chen, and Wei Wang. 2023. Lion: Adversarial distillation of proprietary large language models. *arXiv preprint*

arXiv:2305.12870 (2023).
[36] Nicolai M Josuttis. 2012. The C++ standard library: a tutorial and reference. (2012).
[37] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.
[38] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
[39] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. {InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.
[40] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691* (2021).
[41] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691* (2021).
[42] Suyi Li, Hanfeng Lu, Tianyuan Wu, Minchen Yu, Qizhen Weng, Xusheng Chen, Yizhou Shan, Binhang Yuan, and Wei Wang. 2024. CaraServe: CPU-Assisted and Rank-Aware LoRA Serving for Generative LLM Inference. arXiv:2401.11240 [cs.DC] https://arxiv.org/abs/2401.11240
[43] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190* (2021).
[44] Yunxiang Li, Zihan Li, Kai Zhang, Ruilong Dan, Steve Jiang, and You Zhang. 2023. Chatdoctor: A medical chat model fine-tuned on a large language model meta-ai (llama) using medical domain knowledge. *Cureus* 15, 6 (2023).
[45] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. 2024. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459* (2024).
[46] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 663–679.
[47] Long Lian, Baifeng Shi, Adam Yala, Trevor Darrell, and Boyi Li. 2023. Llm-grounded video diffusion models. *arXiv preprint arXiv:2309.17444* (2023).
[48] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.
[49] Tiedong Liu and Bryan Kian Hsiang Low. 2023. Goat: Fine-tuned llama outperforms gpt-4 on arithmetic tasks. *arXiv preprint arXiv:2305.14201* (2023).
[50] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2021. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602* (2021).
[51] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*. 10012–10022.
[52] Bei Luo, Raymond YK Lau, Chunping Li, and Yain-Whar Si. 2022. A critical review of state-of-the-art chatbot designs and applications. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 12, 1 (2022), e1434.
[53] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems* 36 (2023), 21702–21720.
[54] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. 2022. PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. https://github.com/huggingface/peft.
[55] Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, Iman Mirzadeh, Mahyar Najibi, Dmitry Belenko, Peter Zatloukal, and Mohammad Rastegari. 2024. OpenELM: An Efficient Language Model Family with Open Training and Inference Framework. arXiv:2404.14619 [cs.CL] https://arxiv.org/abs/2404.14619
[56] Liang Mi, Weijun Wang, Wenming Tu, Qingfeng He, Rui Kong, Xinyu Fang, Yazhu Dong, Yikang Zhang, Yunchun Li, Meng Li, et al. 2024. V-LoRA: An Efficient and Flexible System Boosts Vision Applications with LoRA LMM. *arXiv preprint arXiv:2411.00915* (2024).
[57] Microsoft. 2023. DeepSpeed-MII: DeepSpeed Model Implementations for Inference. https://github.com/microsoft/DeepSpeed-MII. Accessed: [Insert date of access].
[58] MLC team. 2023-2025. *MLC-LLM*. https://github.com/mlc-ai/mlc-llm
[59] Sheshera Mysore, Zhuoran Lu, Mengting Wan, Longqi Yang, Steve Menezes, Tina Baghaee, Emmanuel Barajas Gonzalez, Jennifer Neville, and Tara Safavi. 2023. Pearl: Personalizing large language model writing assistants with generation-calibrated retrievers. *arXiv preprint arXiv:2311.09180* (2023).
[60] Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang, et al. 2016. Abstractive text summarization using sequence-to-sequence rnns and beyond. *arXiv preprint arXiv:1602.06023* (2016).
[61] OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774
[62] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2024. An Empirical Study of the Non-determinism of ChatGPT in Code Generation. *ACM Transactions on Software Engineering and Methodology* (Sept. 2024). https://doi.org/10.1145/3697010
[63] Marco Paolieri, Eduardo Quiñones, Francisco J Cazorla, Robert I Davis, and Mateo Valero. 2011. IAˆ 3: An interference aware allocation algorithm for multicore hard real-time systems. In *2011 17th IEEE Real-Time and Embedded Technology and Applications Symposium*. IEEE, 280–290.
[64] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. 2017. Automatic differentiation in PyTorch. (2017).
[65] James L Peterson and Abraham Silberschatz. 1985. *Operating system concepts*. Addison-Wesley Longman Publishing Co., Inc.
[66] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. arXiv:2311.12022 [cs.AI] https://arxiv.org/abs/2311.12022
[67] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
[68] Timo Schick and Hinrich Schütze. 2020. Exploiting cloze questions for few shot text classification and natural language inference. *arXiv preprint arXiv:2001.07676* (2020).
[69] Tianxiang Shen, Ji Qi, Jianyu Jiang, Xian Wang, Siyuan Wen, Xusheng Chen, Shixiong Zhao, Sen Wang, Li Chen, Xiapu Luo, Fengwei Zhang, and Heming Cui. 2022. SOTER: Guarding Black-box Inference for General Neural Networks at the Edge. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 723–738. https://www.usenix.org/conference/atc22/presentation/shen
[70] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. 2023. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285* (2023).
[71] Yu Shu, Siwei Dong, Guangyao Chen, Wenhao Huang, Ruihua Zhang, Daochen Shi, Qiqi Xiang, and Yemin Shi. 2023. Llasm: Large language and speech model. *arXiv preprint arXiv:2308.15930* (2023).
[72] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 590–606.
[73] Felix Stahlberg. 2020. Neural machine translation: A review. *Journal of Artificial Intelligence Research* 69 (2020), 343–418.
[74] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. *arXiv preprint arXiv:2406.03243* (2024).
[75] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. arXiv:2210.09261 [cs.CL] https://arxiv.org/abs/2210.09261
[76] Zhengyang Tang, Xingxing Zhang, Benyou Wang, and Furu Wei. 2024. MathScale: Scaling Instruction Tuning for Mathematical Reasoning. arXiv:2403.02884 [cs.CL] https://arxiv.org/abs/2403.02884
[77] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models. https://crfm. stanford. edu/2023/03/13/alpaca. html* 3, 6 (2023), 7.
[78] Gemini Team. 2024. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805 [cs.CL] https://arxiv.org/abs/2312.11805
[79] Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. 2024. OpenMathInstruct-2: Accelerating AI for Math with Massive Open-Source Instruction Data. *arXiv preprint arXiv:2410.01560* (2024).
[80] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] https://arxiv.org/abs/2302.13971
[81] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
[82] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in neural information processing systems* 33

(2020), 5776–5788.
[83] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. arXiv:2406.01574 [cs.CL] https://arxiv.org/abs/2406.01574
[84] Zihao Wang, Wei Liu, Qian He, Xinglong Wu, and Zili Yi. 2022. Clip-gen: Language-free training of a text-to-image generator with clip. *arXiv preprint arXiv:2203.00386* (2022).
[85] Ziyao Wang, Zheyu Shen, Yexiao He, Guoheng Sun, Hongyi Wang, Lingjuan Lyu, and Ang Li. 2024. FLoRA: Federated Fine-Tuning Large Language Models with Heterogeneous Low-Rank Adaptations. arXiv:2409.05976 [cs.LG] https://arxiv.org/abs/2409.05976
[86] Adhika Pramita Widyassari, Supriadi Rustad, Guruh Fajar Shidik, Edi Noersasongko, Abdul Syukur, Affandy Affandy, et al. 2022. Review of automatic text summarization techniques & methods. *Journal of King Saud University-Computer and Information Sciences* 34, 4 (2022), 1029–1046.
[87] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45. https://www.aclweb.org/anthology/2020.emnlp-demos.6
[88] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. {dLoRA}: Dynamically Orchestrating Requests and Adapters for {LoRA}{LLM} Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 911–927.
[89] Mengzhou Xia, Zexuan Zhong, and Danqi Chen. 2022. Structured pruning learns compact and accurate models. *arXiv preprint arXiv:2204.00408* (2022).
[90] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
[91] Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. 2025. LServe: Efficient Long-sequence LLM Serving with Unified Sparse Attention. arXiv:2502.14866 [cs.CL] https://arxiv.org/abs/2502.14866
[92] Zhilin Yang, Ye Yuan, Yuexin Wu, William W Cohen, and Russ R Salakhutdinov. 2016. Review networks for caption generation. *Advances in neural information processing systems* 29 (2016).
[93] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2024. A Survey on Multimodal Large Language Models. *National Science Review* (Nov. 2024). https://doi.org/10.1093/nsr/nwae403
[94] Shengbin Yue, Wei Chen, Siyuan Wang, Bingxuan Li, Chenchen Shen, Shujun Liu, Yuxuan Zhou, Yao Xiao, Song Yun, Xuanjing Huang, et al. 2023. Disc-lawllm: Fine-tuning large language models for intelligent legal services. *arXiv preprint arXiv:2309.11325* (2023).
[95] Boxiang Yun, Yan Wang, Jieneng Chen, Huiyu Wang, Wei Shen, and Qingli Li. 2021. Spectr: Spectral transformer for hyperspectral pathology image segmentation. *arXiv preprint arXiv:2103.03604* (2021).
[96] Duzhen Zhang, Yahan Yu, Jiahua Dong, Chenxing Li, Dan Su, Chenhui Chu, and Dong Yu. 2024. MM-LLMs: Recent Advances in MultiModal Large Language Models. arXiv:2401.13601 [cs.CL] https://arxiv.org/abs/2401.13601
[97] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512* (2023).
[98] Yue Zhang, Leyang Cui, Deng Cai, Xinting Huang, Tao Fang, and Wei Bi. 2023. Multi-task instruction tuning of llama for specific scenarios: A preliminary study on writing assistance. *arXiv preprint arXiv:2305.13225* (2023).
[99] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] https://arxiv.org/abs/2312.07104
[100] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. arXiv:2311.07911 [cs.CL] https://arxiv.org/abs/2311.07911

A Artifact Appendix

A.1 Abstract

This artifact provides a complete workflow to reproduce the performance evaluation of EdgeLoRA, a multi-tenant LLM serving system optimized for edge devices. The source code is publicly available at https://github.com/shenzheyu/EdgeLoRA.git, with precompiled binaries hosted online for convenience. The artifact supports deployment on Jetson AGX Orin, Jetson Orin Nano, and Raspberry Pi 5 devices, and includes detailed installation and execution instructions.

To reproduce the results in the paper, users can launch the EdgeLoRA server with two arguments specifying the model and the number of LoRA adapters, followed by an experiment script that simulates synthetic workloads. Metrics such as throughput, average request latency, first-token latency, and SLO attainment are automatically reported. The default experiment replicates the results presented in the paper, while the setup can be easily customized via parameters such as request rate, adapter count, and input/output lengths. The full experiment completes within minutes and requires approximately 20GB of disk space.

By following the provided steps, users can replicate the benchmark results or conduct customized experiments to evaluate EdgeLoRA's scalability and efficiency across a wide range of configurations.

A.2 Artifact check-list (meta-information)

- **Compilation:** `gcc/g++`, `nvcc`
- **Binary:** Precompiled EdgeLoRA binary available at https://github.com/shenzheyu/EdgeLoRA/releases/tag/v1.0.0
- **Hardware:** Jetson Agx Orin Developer Kit, Jetson Orin Nano, and Rasperry Pi 5
- **Metrics:** Throughput, average request latency, average first-token latency, and SLO attainment.
- **Output:** Printed summary of performance metrics to terminal
- **Experiments:** Described below
- **How much disk space required (approximately)?:** 20GB
- **How much time is needed to prepare workflow (approximately)?:** 1 hour
- **How much time is needed to complete experiments (approximately)?:** 10 minutes per configuration
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT License
- **Archived (provide DOI)?:** 10.6084/m9.figshare.28675676

A.3 Description

A.3.1 How to access.

- Source code: https://github.com/shenzheyu/EdgeLoRA.git
- Binary release: https://github.com/shenzheyu/EdgeLoRA/releases/tag/v1.0.0

A.3.2 Hardware dependencies. To match the experiment setup described in the paper, EdgeLoRA is evaluated on the following devices:

- Jetson AGX Orin Developer Kit
- Jetson Orin Nano
- Raspberry Pi 5

A.3.3 Software dependencies.

- Ubuntu 22.04
- L4T Driver Package Version: 36.6.3
- JetPack Version: 6.2
- g++ Compiler: 11.4.0
- Node.js: 20.18.3

A.4 Installation

The following steps describe how to install EdgeLoRA from source:

```
# clone the EdgeLoRA repository
git clone https://github.com/shenzheyu/EdgeLoRA.git

# compile the source code
cd EdgeLoRA/edgelora
export GGML_CUDA=1 # enable CUDA if device has a GPU
make llama-server

# download pre-trained models and adapters
pip install gdown
gdown https://drive.google.com/uc\?id\=1
      cyU2MUe8V4bo4IuKZG7cEZ2wpxJzD0nn
tar -xzvf models.tar.gz

# install the dependencies of experiment script
cd llama-client
npm install gamma progress
```

A.5 Experiment workflow

To reproduce the default experiment for EdgeLoRA using the Llama3.1-8B model and 20 LoRA adapters:

```
# launch the EdgeLoRA server
bash server.sh Llama3.1-8B 20

# run the default experiment script
cd llama-client
node edge_lora.js
```

The script prints the resulting throughput, average request latency, first-token latency, and SLO attainment directly to the terminal.

A.6 Evaluation and expected results

The server should launch successfully, and the initial terminal output should indicate that all slots are idle. After running the experiment, performance metrics should be displayed. These results are expected to be consistent with the values reported in the paper, validating the correctness of the artifact setup.

A.7 Experiment customization

The server can be launched using the following command with two arguments: `bash server.sh <model> <lora_count>`.

- **model**: Specifies the name of the base language model to be served. Supported options include `OpenELM-1.1B`, `Llama3.2-3B`, and `Llama3.1-8B`.
- **lora_count**: Indicates the total number of LoRA adapters to be managed by the server. This value can range from a few dozen to several thousand.

The above experiment script could also be customized with multiple arguments in the 'llama-client/edge_lora.js' file:

- **n**: Number of LoRA adapters available in the system. Controls adapter diversity.
- **alpha**: Power-law exponent that defines the skewness of request distribution across adapters.
- **R**: Total request rate, i.e., how many requests per second are sent across all adapters.
- **cv**: Coefficient of variance for arrival intervals in the Gamma process, defining burstiness of the workload.
- **traceDuration**: Duration of the synthetic trace (in milliseconds), default representing 5 minutes.
- **Il, Iu**: Lower and upper bounds for input token lengths sampled from a uniform distribution.
- **Ol, Ou**: Lower and upper bounds for output token lengths, also sampled uniformly.

A.8 Notes

- The `edgelora_wo_aas` folder contains the implementation of EdgeLoRA without adaptive adapter selection. Its usage is similar to the standard EdgeLoRA workflow.
- The `adapter-router` folder provides the implementation for fine-tuning and evaluating the adapter router. This component requires a custom version of the HuggingFace Transformers library, which can be installed using:

```
pip install git+https://github.com/shenzheyu/transformers.
      git@edgelora#egg=transformers
```